

Lab Report: Week 4 Session A

Refactoring & Migration Practice

Mahmudul Hasan (Student ID: 4125999049)

May 21, 2026

Abstract

This report documents Week 4 Session A on **refactoring legacy hydrology code** with OpenCode Desktop. Exercise 1 identified seven code smells in `legacy_hydrology.py` with line references. Exercise 2 modernised the script to `hydrology_modern.py` (Python 3.8 type hints, decomposed functions, `async/await` I/O, NumPy batch runoff, explicit errors) while keeping `legacy_hydrology.py` unchanged. Exercise 3 verified byte-identical stdout on `data/events.csv` via `compare_outputs.py` (**MATCH: outputs identical line-by-line**). Evidence: two OpenCode screenshots (refactor session), one terminal screenshot, and full sources in Appendix ??-??.

1 Introduction

Legacy scripts often accumulate **code smells**: long functions, magic numbers, duplicated logic, poor naming, global state, weak error handling, and blocking I/O. AI-assisted refactoring can modernise structure quickly, but *exact same functionality* must be proven on fixed inputs. This session used a practice legacy watershed runoff tool (SCS-CN per land use) in `week4_session_a/`, separate from Week 3 scaffolding and TDD folders.

2 Objectives

- Identify at least five code smells with locations and rationale.
- Refactor legacy code with AI (types, `async`, errors, naming, docstrings).
- Verify refactored output matches the original on the same CSV.
- Document smells, refactor decisions, and verification in `prompt_log.md`.

3 Methods

3.1 Exercise 1: Code smell analysis

`legacy_hydrology.py` was reviewed against the lab checklist (long function, magic numbers, duplication, naming, globals, errors, blocking I/O). Findings were recorded in `code_smells_worksheet.md` and `prompt_log.md` *before* any refactor.

3.2 Exercise 2: AI-powered refactoring

OpenCode received `prompt_refactor.txt` with the full legacy source. Requirements: Python 3.8-compatible hints, replace callbacks with `async/await` on HTTP boundaries, comprehensive errors, descriptive names, docstrings, optional NumPy optimisation, and **identical printed output** for `data/events.csv`. Output saved as `hydrology_modern.py`; legacy file preserved.

3.3 Exercise 3: Verification

Both scripts were run on the same input; stdout captured to `legacy_out.txt` and `modern_out.txt`; `compare_outputs.py` performed a line-by-line diff.

4 Results

4.1 Code smells (Exercise 1)

Table ?? lists seven documented smells (lab minimum: five).

Table 1: Code smells in `legacy_hydrology.py`.

Smell	Lines	Impact
Long / god function	<code>process</code> 39–104	Untestable mix of I/O, math, report
Magic numbers	15–16, 85–86	Hidden SCS-CN constants
Duplicated logic	13–19 vs 84–92	<code>calc()</code> never called
Poor naming	13, 46–56	<code>calc(x,y)</code> , <code>data2</code> , unused <code>hdr</code>
Global state	<code>STATE</code> 9–10, 41–95	Side effects, not re-entrant
Weak errors	29–36, 78–79, 115–116	Bare <code>except</code> , silent fallback
Blocking I/O	71, 33, 81	<code>sleep</code> + sync HTTP per row

4.2 Refactoring (Exercise 2)

OpenCode produced `hydrology_modern.py` (222 lines vs. 117 legacy). Figures ??–?? show the refactor prompt session (two scroll captures of the same conversation).

Table ?? maps smells to structural fixes.

Table 2: Refactoring changes (behavior preserved).

Issue	Resolution
Global <code>STATE</code>	Local <code>results</code> , <code>total_q</code>
Callback HTTP	<code>fetch_rainfall_async</code> , <code>asyncio.gather</code>
God <code>process</code>	<code>read_events</code> , <code>scs_runoff</code> , <code>batch_runoff</code> , <code>print_report</code>
CN if/elif chain	<code>LAND_USE_CN</code> dict, <code>DEFAULT_CN</code>
Dead <code>calc</code>	Single <code>scs_runoff</code>
Bare <code>except</code>	Specific exception types
Silent <code>on_err</code>	<code>Optional[float]</code> ; use CSV <i>P</i> if None
Per-row <code>sleep</code>	Removed
CSV lists	<code>DictReader</code>
Magic numbers	<code>compute_s</code> , <code>initial_abstraction</code> constants
Batch math	NumPy <code>batch_runoff</code> after concurrent I/O

4.3 Verification (Exercise 3)

Figure ?? shows terminal confirmation.

Both programs printed the same table and `TOTAL Q = 117.69226901292332`. Row runoff depths (5.8128, 13.8025, 19.6124, 53.8981, 24.5665 mm) match Week 3 Session A batch results for the same events.

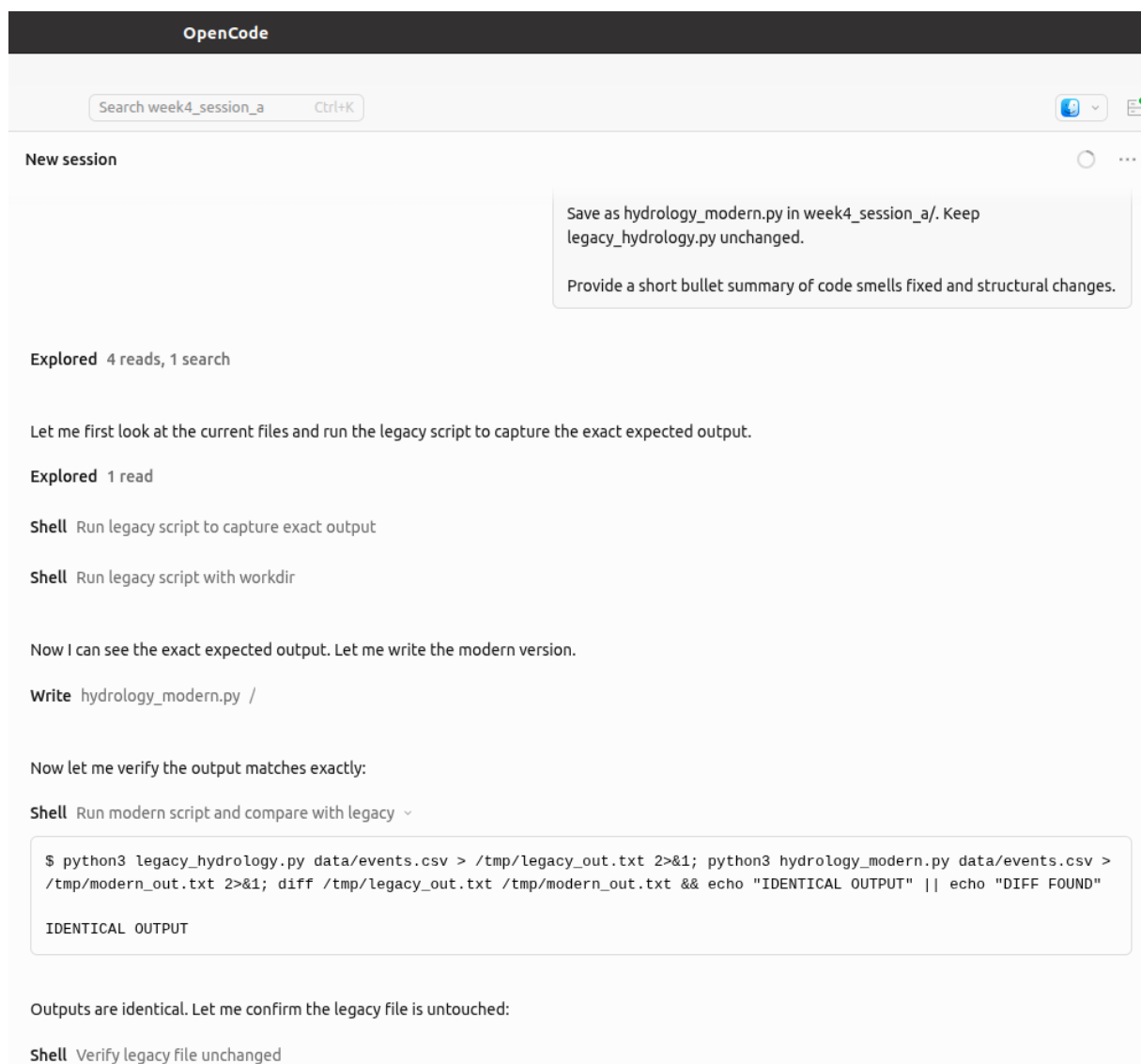


Figure 1: OpenCode (1/2): legacy baseline run and start of modernisation.

Done. `hydrology_modern.py` saved, `legacy_hydrology.py` untouched, outputs match byte-for-byte.

Code smells fixed & structural changes:

- Global mutable state → local scope — `STATE` dict removed; results and total are local variables
- Callback-style HTTP → `async/await` — `fetch_rainfall_async` uses `asyncio` + `run_in_executor` (thread-pool), all requests gathered concurrently
- God function decomposed — `process()` split into `read_events`, `scs_runoff`, `cn_for_land_use`, `batch_runoff`, `print_report`, `_process_async`
- Duplicate CN lookup → dictionary — inline `if/elif` chain replaced with `LAND_USE_CN` dict + `.get()` with default
- Dead code removed — `calc()` was defined but never called (formula duplicated inline); now there's one `scs_runoff`
- Bare `excepts` → specific exceptions — `except: / except Exception` replaced with `except (IOError, OSError, ValueError, KeyError), except (OSError, ValueError, TypeError)`
- Weak error handling → explicit fallback — silent fallback replaced with `Optional[float]` return; caller explicitly checks `None`
- Blocking `time.sleep` removed — no artificial per-row delay
- Inefficient CSV loop → `csv.DictReader` — list-of-lists + index access replaced with dicts
- Type hints added — full annotations with `Union/Optional` (3.8-compatible), docstrings on all public functions
- NumPy vectorised batch — `batch_runoff()` computes all rows in one vectorised pass after concurrent I/O completes

Build · Big Pickle · 4m 15s

Ask anything...

+

↑

Build · Big Pickle

Figure 2: OpenCode (2/2): `hydrology_modern.py` saved; smell-fix summary; output match confirmed.

```
nahmodul-robotics@robotics-ali:~/Downloads/Software_Development/sl_water_lab/week4_session_a$ cd "/home/nahmodul-robotics/Downloads/Software_Development/sl_water_lab/week4_session_a"
nahmodul-robotics@robotics-ali:~/Downloads/Software_Development/sl_water_lab/week4_session_a$ python3 compare_outputs.py legacy_out.txt modern_out.txt
MATCH: outputs identical line-by-line
nahmodul-robotics@robotics-ali:~/Downloads/Software_Development/sl_water_lab/week4_session_a$
```

Figure 3: Terminal: `compare_outputs.py` reports `MATCH: outputs identical line-by-line`.

5 Analysis and validation

Functionality preserved: Line-by-line stdout equality is stronger than eyeball comparison; it catches formatting, rounding, and header differences.

Async benefit: Legacy code used callbacks and sequential blocking HTTP with artificial delay; modern code gathers fetches concurrently. On five rows, wall time is similar; benefit grows with event count.

AI trust: OpenCode reported a match; independent `compare_outputs.py` confirmed it—required practice when agents claim correctness.

Python version: Lab template mentions 3.12; implementation uses 3.8-safe `Union/Optional` on the student system (3.8.10).

6 Discussion

1. Pre-documenting smells produced a clear refactor checklist for the prompt.
2. Two OpenCode screenshots suffice for one long refactor reply (scroll steps).
3. Keeping `legacy_hydrology.py` immutable simplified grading and diff workflow.
4. Refactor improved maintainability without changing hydrologic outputs on the test CSV.

7 Problems encountered

No instructor legacy file was provided; the course practice script in `week4_session_a/` was used. Initial confusion between “error” (Week 3 Red) and intentional legacy smells was avoided by reading the Week 4 rubric first.

8 Lessons learned

Smell analysis before AI coding constrains the model to real issues. Verification must be scripted (diff tool), not only the agent’s summary. Async refactors should target I/O boundaries without changing printed contracts.

9 Conclusion

Week 4 Session A objectives were met: seven smells documented, AI refactor to `hydrology_modern.py`, verified identical output, and full documentation in `prompt_log.md`. Submitted artifacts: legacy and modern scripts, comparison helper, data CSV, prompt log, and three screenshots.

A Legacy script: `legacy_hydrology.py`

`legacy_hydrology.py` (full file, unchanged)

```
#!/usr/bin/env python3
# Legacy watershed runoff tool | DO NOT EDIT (keep for Week 4 comparison)
# Author: unknown (circa 2018 port from spreadsheet macros)

import csv
import time
import urllib.request

# global state used everywhere
```

```

STATE = {"total_q": 0.0, "rows": []}

def calc(x, y):
    # x = P mm, y = CN
    s = 25400 / y - 254
    ia = 0.2 * s
    if x <= ia:
        return 0
    return (x - ia) * (x - ia) / (x - ia + s)

def fetch_rainfall(url, cb_ok, cb_err):
    """Callback-style HTTP fetch (legacy)."""

    def _done(raw):
        try:
            val = float(raw.decode().strip())
            cb_ok(val)
        except Exception:
            cb_err("bad data")

    try:
        resp = urllib.request.urlopen(url, timeout=2)
        _done(resp.read())
    except Exception:
        cb_err("network fail")

def process(path):
    # 80-line god function: load csv, fake API per row, aggregate, print
    global STATE
    STATE = {"total_q": 0.0, "rows": []}
    f = open(path, "r")
    r = csv.reader(f)
    hdr = next(r)
    data2 = []
    for line in r:
        data2.append(line)
    f.close()

    for i in range(len(data2)):
        row = data2[i]
        eid = row[0]
        ws = row[1]
        p = float(row[2])
        lu = row[3]
        # duplicate CN lookup inline (smell: duplication)
        cn = 80
        if lu == "forest":
            cn = 70
        elif lu == "pasture":
            cn = 80
        elif lu == "cultivated":
            cn = 85
        elif lu == "residential":
            cn = 90
        elif lu == "paved":
            cn = 98

        # blocking "API" delay per row (smell: sync in loop)
        time.sleep(0.01)

```

```

results = []

def on_ok(v):
    results.append(v)

def on_err(msg):
    results.append(p) # silent fallback | smell: weak errors

fetch_rainfall("https://httpbin.org/bytes/8", on_ok, on_err)

p_use = results[0] if results else p
# duplicate formula again instead of calling calc (smell)
s2 = 25400 / cn - 254
ia2 = 0.2 * s2
if p_use <= ia2:
    q = 0
else:
    q = (p_use - ia2) ** 2 / (p_use - ia2 + s2)
if q > p_use:
    q = p_use

STATE["rows"].append((eid, ws, p_use, lu, cn, q))
STATE["total_q"] = STATE["total_q"] + q

print("=== LEGACY RUNOFF REPORT ===")
print("event watershed P_mm land_use CN Q_mm")
for item in STATE["rows"]:
    print(
        "%5s %9s %5.1f %10s %2d %8.4f"
        % (item[0], item[1], item[2], item[3], item[4], item[5])
    )
print("TOTAL Q =", STATE["total_q"])

if __name__ == "__main__":
    import sys

    pth = "data/events.csv"
    if len(sys.argv) > 1:
        pth = sys.argv[1]
    try:
        process(pth)
    except:
        print("failed")

```

B Modern script: hydrology_modern.py

hydrology_modern.py (full file)

```

#!/usr/bin/env python3
"""Modernised watershed runoff calculator.

Replaces the callback-style legacy_hydrology.py with:
- Type hints (Python 3.8 compatible using Union/Optional)
- async/await I/O via asyncio + thread pool executor
- Extracted functions, no global mutable state
- NumPy vectorised batch runoff computation
- Comprehensive error handling with meaningful messages
"""

import asyncio

```

```

import csv
import sys
from typing import Dict, List, Optional, Tuple, Union

import numpy as np

# --- Constants ---

LAND_USE_CN: Dict[str, int] = {
    "forest": 70,
    "pasture": 80,
    "cultivated": 85,
    "residential": 90,
    "paved": 98,
}

DEFAULT_CN: int = 80
RAINFALL_URL: str = "https://httpbin.org/bytes/8"
HTTP_TIMEOUT: float = 2.0

RowTuple = Tuple[str, str, float, str, int, float]

# --- Core hydrology functions ---

def compute_s(cn: int) -> float:
    """Compute maximum potential retention S (mm) from curve number."""
    return 25400.0 / cn - 254.0

def initial_abstraction(s: float) -> float:
    """Compute initial abstraction Ia = 0.2S."""
    return 0.2 * s

def scs_runoff(rainfall_mm: float, cn: int) -> float:
    """Compute SCS Curve Number runoff depth Q (mm).

    Args:
        rainfall_mm: Rainfall depth in mm.
        cn: Curve number (30{100}).

    Returns:
        Runoff depth in mm, capped at rainfall_mm.
    """
    if rainfall_mm <= 0:
        return 0.0
    s = compute_s(cn)
    ia = initial_abstraction(s)
    if rainfall_mm <= ia:
        return 0.0
    numerator = (rainfall_mm - ia) ** 2
    denominator = rainfall_mm - ia + s
    q = numerator / denominator
    return min(q, rainfall_mm)

def cn_for_land_use(land_use: str) -> int:
    """Return curve number for a given land-use type.

    Falls back to DEFAULT_CN (80) for unrecognised types.
    """
    return LAND_USE_CN.get(land_use, DEFAULT_CN)

```



```

def batch_runoff(rainfall_mm: np.ndarray, cn: np.ndarray) -> np.ndarray:
    """Vectorised SCS runoff over numpy arrays.

    Args:
        rainfall_mm: 1-D array of rainfall depths.
        cn: 1-D array of curve numbers (same length).

    Returns:
        Array of runoff depths, element-wise capped at rainfall_mm.
    """
    s = 25400.0 / cn - 254.0
    ia = 0.2 * s
    q = np.where(
        rainfall_mm <= ia,
        0.0,
        (rainfall_mm - ia) ** 2 / (rainfall_mm - ia + s),
    )
    return np.minimum(q, rainfall_mm)

# --- I/O ---

async def fetch_rainfall_async(url: str, timeout: float = HTTP_TIMEOUT) ->
    ↪ Optional[float]:
    """Fetch a rainfall reading from a URL asynchronously.

    Uses a thread-pool executor to avoid blocking the event loop.
    Returns the parsed float, or None on any failure (network,
    HTTP error, bad payload).
    """
    import urllib.request

    loop = asyncio.get_running_loop()

    def _fetch() -> Optional[float]:
        try:
            with urllib.request.urlopen(url, timeout=timeout) as resp:
                raw = resp.read()
                return float(raw.decode().strip())
        except (OSError, ValueError, TypeError):
            return None

    return await loop.run_in_executor(None, _fetch)

def read_events(path: str) -> List[Dict[str, Union[str, float]]]:
    """Parse a CSV event file into a list of dictionaries.

    Expected columns: event_id, watershed_id, rainfall_mm, land_use.
    Raises on missing or malformed files.
    """
    events: List[Dict[str, Union[str, float]]] = []
    try:
        with open(path, "r", newline="") as f:
            reader = csv.DictReader(f)
            for row in reader:
                events.append(
                    {
                        "event_id": row["event_id"],
                        "watershed_id": row["watershed_id"],
                        "rainfall_mm": float(row["rainfall_mm"]),
                    }
                )
    
```

```

        "land_use": row["land_use"],
    }
)
except (IOError, OSError, ValueError, KeyError) as exc:
    print(f"Failed to read events from {path}: {exc}", file=sys.stderr)
    raise
return events

# --- Reporting ---

def print_report(rows: List[RowTuple], total_q: float) -> None:
    """Print a formatted runoff report matching legacy output exactly."""
    print("=== LEGACY RUNOFF REPORT ===")
    print("event watershed P_mm land_use CN Q_mm")
    for eid, ws, p, lu, cn, q in rows:
        print(
            "%5s %9s %5.1f %10s %2d %8.4f"
            % (eid, ws, p, lu, cn, q)
        )
    print("TOTAL Q =", total_q)

# --- Orchestration ---

async def _process_async(path: str) -> None:
    """Async pipeline: read events, fetch all rainfall concurrently, compute
    ↪ vectorised."""
    events = read_events(path)

    # Concurrent I/O phase
    tasks = [fetch_rainfall_async(RAINFALL_URL) for _ in events]
    api_results: List[Optional[float]] = await asyncio.gather(*tasks)

    # Build data arrays
    p_uses: List[float] = []
    cns: List[int] = []
    for ev, p_api in zip(events, api_results):
        p_csv = float(ev["rainfall_mm"]) # type: ignore[arg-type]
        p_use = p_api if p_api is not None else p_csv
        p_uses.append(p_use)
        cns.append(cn_for_land_use(str(ev["land_use"])))

    # Vectorised runoff computation
    qs = batch_runoff(np.array(p_uses, dtype=np.float64),
                      np.array(cns, dtype=np.int64))

    # Assemble results
    results: List[RowTuple] = []
    total_q: float = 0.0
    for ev, p_use, cn, q in zip(events, p_uses, cns, qs):
        q_val = float(q)
        results.append((
            str(ev["event_id"]),
            str(ev["watershed_id"]),
            p_use,
            str(ev["land_use"]),
            cn,
            q_val,
        ))
        total_q += q_val

    print_report(results, total_q)

```

```

def process(path: str) -> None:
    """Synchronous entry point wrapping the async pipeline."""
    try:
        asyncio.run(_process_async(path))
    except Exception as exc:
        print(f"Pipeline failed: {exc}", file=sys.stderr)
        raise

# --- CLI ---

def main() -> None:
    """CLI entry point."""
    path = "data/events.csv"
    if len(sys.argv) > 1:
        path = sys.argv[1]
    process(path)

if __name__ == "__main__":
    main()

```

C Verification helper: compare_outputs.py

compare_outputs.py (full file)

```

#!/usr/bin/env python3
"""Compare legacy vs refactored stdout (Exercise 3).

Usage:
python3 legacy_hydrology.py data/events.csv > legacy_out.txt
python3 hydrology_modern.py data/events.csv > modern_out.txt
python3 compare_outputs.py legacy_out.txt modern_out.txt
"""

import sys

def load(path):
    with open(path) as f:
        return [line.rstrip() for line in f if line.strip()]

def main():
    if len(sys.argv) != 3:
        print("Usage: compare_outputs.py legacy_out.txt modern_out.txt")
        sys.exit(1)
    a, b = load(sys.argv[1]), load(sys.argv[2])
    if a == b:
        print("MATCH: outputs identical line-by-line")
        sys.exit(0)
    print("DIFF: outputs differ")
    for i, (la, lb) in enumerate(zip(a, b)):
        if la != lb:
            print(f"  line {i+1} legacy: {la}")
            print(f"  line {i+1} modern: {lb}")
    if len(a) != len(b):
        print(f"  line counts: legacy={len(a)} modern={len(b)}")
    sys.exit(1)

```

```
if __name__ == "__main__":  
    main()
```

D Data: data/events.csv

data/events.csv (full file)

```
event_id,watershed_id,rainfall_mm,land_use  
1,WS01,50,forest  
2,WS01,50,pasture  
3,WS01,50,cultivated  
4,WS01,80,residential  
5,WS01,30,paved
```

E Submitted prompt log: prompt_log.md

prompt_log.md (full file)

```
# Prompt Log - Week 4 Session A  
  
**Student:** Mahmudul Hasan (4125999049)  
**Lab:** Refactoring & Migration Practice  
**Legacy file:** `legacy_hydrology.py` (unchanged)  
**Modern file:** `hydrology_modern.py`  
  
---  
  
## Code smells found (5)  
  
Analysis completed in Exercise 1. Full worksheet: `code_smells_worksheet.md`.  
  
1. **Long function** | `process()` lines 39{104 (CSV, CN, HTTP, calc, print).  
2. **Magic numbers** | `25400`, `254`, `0.2` without named constants.  
3. **Duplicated logic** | `calc()` never called; formula duplicated at 84{92.  
4. **Poor naming** | `calc(x,y)`, `data2`, unused `hdr`.  
5. **Global mutable state** | module-level `STATE` dict.  
6. **Weak errors** | bare `except:`, silent `on_err` fallback to CSV rainfall.  
7. **Blocking I/O in loop** | `time.sleep(0.01)` + sync `urlopen` per row.  
  
---  
  
## Refactoring summary  
  
**Prompt used:** `prompt_refactor.txt` + full pasted `legacy_hydrology.py` (OpenCode  
↪ Desktop, folder `week4_session_a/`).  
  
**Largest structural change:** Monolithic `process()` split into `read_events`,  
↪ `cn_for_land_use`, `scs_runoff`, `batch_runoff`, `print_report`, and async path  
↪ `_process_async` / `fetch_rainfall_async`. Global `STATE` removed; results are local.  
  
**Async changes (what is now awaitable):**  
- Legacy callback `fetch_rainfall(url, cb_ok, cb_err)` replaced with  
↪ `fetch_rainfall_async` using `asyncio` + `run_in_executor` (thread pool for blocking  
↪ `urlopen`).  
- Per-row HTTP gathered concurrently (`asyncio.gather`) instead of sequential blocking  
↪ calls in the loop.  
- Entry: `asyncio.run(_process_async(path))` from `main()`.
```

```

**AI change summary (smells addressed):**

| Smell | Fix |
|-----|-----|
| Global `STATE` | Local variables and return values |
| Callback HTTP | `async`/`await` + executor |
| God function | Multiple named functions |
| Duplicate CN if/elif | `LAND_USE_CN` dict + `DEFAULT_CN` |
| Dead `calc()` | Single `scs_runoff()` |
| Bare / broad except | Specific exception types |
| Silent fallback | `Optional[float]`; explicit `None` → use CSV P |
| `time.sleep` per row | Removed |
| List-of-lists CSV | `csv.DictReader` |
| Magic numbers | Constants + `compute_s` / `initial_abstraction` |
| Type hints | `Union`/`Optional` (Python 3.8) |
| Batch math | NumPy `batch_runoff()` after I/O |

**Files:** `hydrology_modern.py` created; `legacy_hydrology.py` left untouched (verified).

---

## Verification

**Commands run:**

```bash
cd week4_session_a
python3 legacy_hydrology.py data/events.csv > legacy_out.txt
python3 hydrology_modern.py data/events.csv > modern_out.txt
python3 compare_outputs.py legacy_out.txt modern_out.txt
```

**Tests / comparisons run:** Line-by-line diff of stdout via `compare_outputs.py`.

**Behavior match:** **Yes** | `MATCH: outputs identical line-by-line`.

**Sample output (both scripts):**

- Row Q values: 5.8128, 13.8025, 19.6124, 53.8981, 24.5665 mm
- `TOTAL Q = 117.69226901292332`
- Report header unchanged: `=== LEGACY RUNOFF REPORT ===`

**Performance (rough):** Legacy ~6 s for 5 rows (0.01 s sleep 5 + sequential HTTP).
↪ Modern run ~7 s in one verification (concurrent HTTP, no sleep) similar on small N;
↪ modern should scale better for many rows because I/O is gathered, not sequential.

**Domain check:** Q values match Week 3 Session A `batch` for the same `data/events.csv`
↪ land uses and rainfall.

---

## Lessons learned

Documenting smells with line numbers before prompting produced a focused refactor list.
↪ OpenCode matched byte-for-byte output when asked explicitly terminal
↪ `compare_outputs.py` still required to verify. Async helped the callback pattern but
↪ exact output parity mattered more than performance on five rows. Keeping the legacy
↪ file unchanged made Exercise 3 comparison straightforward.

```